

Genomic plots

Katherine I. Cooper

2026-05-06

Genomic plots for comparison

A few genes have vastly different mean CpG counts: *NRL*, *THRB*.

GOAL: Look at the sequences for CpG islands

Step 1: Getting the sequences for 1,000 bp

```
#Creating a dataframe of the genes
Genes.of.int <- data.frame (
  "Human" = c("THRB", "NRL"),
  "Mouse" = c("Thrb", "Nr1"))

#Mapping the gene symbols to the Ensembl
Ensembl_names.Hs <- mapIds(org.Hs.eg.db,
                           keys = Genes.of.int$Human,
                           keytype = "SYMBOL",
                           column = "ENSEMBL")

## 'select()' returned 1:many mapping between keys and columns
```

```
Ensembl_names.Mm <- mapIds(org.Mm.eg.db,
                           keys = Genes.of.int$Mouse,
                           keytype = "SYMBOL",
                           column = "ENSEMBL")
```

```
## 'select()' returned 1:1 mapping between keys and columns
```

```
#Adding to the Df.
Genes.of.int$Ensembl.Mm <- Ensembl_names.Mm
Genes.of.int$Ensembl.Hs <- Ensembl_names.Hs

head(Genes.of.int)
```

```
##   Human Mouse      Ensembl.Mm      Ensembl.Hs
## 1  THRB  Thrb ENSMUSG00000021779 ENSG00000151090
## 2   NRL   Nr1 ENSMUSG00000040632 ENSG00000129535
```

I will be using the 1kb flanking region around the canonical start site.

```

#Using Ensembl mart archive for a stable connection:
ensembl.hs <- useMart("ensembl",
                     dataset = "hsapiens_gene_ensembl",
                     host = "https://may2025.archive.ensembl.org")

#Acquiring the TSS info
tss_info.Hs.THRB <- getBM(filters = "ensembl_gene_id",
                          values = Genes.of.int$Ensembl.Hs[1],
                          attributes = c("chromosome_name", "transcription_start_site", "strand"),
                          mart = ensembl.hs)

#Getting the canonical start for THRB
tss_info.Hs.THRB <- tss_info.Hs.THRB[1, ]

#Creating the range...THRB seems to be reverse so the start and end are flipped.
start <- tss_info.Hs.THRB$transcription_start_site +1000
end <- tss_info.Hs.THRB$transcription_start_site - 1000

#Flipping start and end...
tss_range.Hs.THRB <- GRanges(seqnames = paste0("chr", tss_info.Hs.THRB$chromosome_name),
                             ranges = IRanges(start = end, end = start),
                             strand = ifelse(tss_info.Hs.THRB$strand == 1, "+", "-"))

remove(start)
remove(end)
head(tss_range.Hs.THRB)

```

Starting with human

```

## GRanges object with 1 range and 0 metadata columns:
##      seqnames      ranges strand
##      <Rle>        <IRanges> <Rle>
## [1]      chr3 24493822-24495822      -
## -----
##      seqinfo: 1 sequence from an unspecified genome; no seqlengths

```

Getting the BS DNA

```

DNA.Hs.THRB <- getSeq(BSgenome.Hsapiens.UCSC.hg38, tss_range.Hs.THRB)

head(DNA.Hs.THRB)

```

```

## DNAStringSet object of length 1:
##      width seq
## [1] 2001 GGTCGACCGAGCGAGGCCCAAGAAGAGCCAGA...AGGATTTCCTCAATGACCATTGTGGGATTATTAT

```

Repeating for the other four... I do not want to make a loop. I am sorry :(

```

#Acquiring the TSS info
tss_info.Hs.NRL <- getBM(filters = "ensembl_gene_id",

```

```

        values = Genes.of.int$Ensembl.Hs[2],
        attributes = c("chromosome_name", "transcription_start_site", "strand"),
        mart = ensembl.hs)

#Getting the canonical start for NRL
tss_info.Hs.NRL <- tss_info.Hs.NRL[1, ]

#Creating the range...THRB seems to be reverse so the start and end are flipped.
start <- tss_info.Hs.NRL$transcription_start_site +1000
end <- tss_info.Hs.NRL$transcription_start_site - 1000

#Flipping start and end...
tss_range.Hs.NRL <- GRanges(seqnames = paste0("chr", tss_info.Hs.NRL$chromosome_name),
                             ranges = IRanges(start = end, end = start),
                             strand = ifelse(tss_info.Hs.NRL$strand == 1, "+", "-"))

remove(start)
remove(end)

DNA.Hs.NRL <- getSeq(BSgenome.Hsapiens.UCSC.hg38, tss_range.Hs.NRL)
head(DNA.Hs.NRL)

```

```

## DNASTringSet object of length 1:
##      width seq
## [1] 2001 ACCCTCCCTAGGTAAGAACCACTCTTACTGGGTA...TGATCCTCGCCCTGGCTGGGTGGGGACGTGTGG

```

```

ensembl.mm <- useMart("ensembl",
                     dataset = "mmusculus_gene_ensembl",
                     host = "https://dec2021.archive.ensembl.org")

#Acquiring the TSS info
tss_info.Mm.Thrb <- getBM(filters = "ensembl_gene_id",
                          values = Genes.of.int$Ensembl.Mm[1],
                          attributes = c("chromosome_name", "transcription_start_site", "strand"),
                          mart = ensembl.mm)

#Getting the canonical start for THRB
tss_info.Mm.Thrb <- tss_info.Mm.Thrb[1, ]

#Creating the range...THRB seems to be reverse so the start and end are flipped.
start <- tss_info.Mm.Thrb$transcription_start_site +1000
end <- tss_info.Mm.Thrb$transcription_start_site - 1000

#Flipping start and end...
tss_range.Mm.Thrb <- GRanges(seqnames = paste0("chr", tss_info.Mm.Thrb$chromosome_name),
                             ranges = IRanges(start = end, end = start),
                             strand = ifelse(tss_info.Mm.Thrb$strand == 1, "+", "-"))

remove(start)
remove(end)

```

```
DNA.Mm.Thrb <- getSeq(BSgenome.Mmusculus.UCSC.mm39, tss_range.Mm.Thrb)
head(DNA.Mm.Thrb)
```

Now for mouse!

```
## DNASTringSet object of length 1:
##      width seq
## [1] 2001 TCCCATGAACTGATAATCACCACGGTCCTCAGG...ATTCCCTCTTTGGGATAGCGGGAGGAAGATCCC
```

NRL

```
#Acquiring the TSS info
tss_info.Mm.Nrl <- getBM(filters = "ensembl_gene_id",
                        values = Genes.of.int$Ensembl.Mm[2],
                        attributes = c("chromosome_name", "transcription_start_site", "strand"),
                        mart = ensembl.mm)

#Getting the canonical start for NRL
tss_info.Mm.Nrl <- tss_info.Mm.Nrl[1, ]

#Creating the range...THRB seems to be reverse so the start and end are flipped.
start <- tss_info.Mm.Nrl$transcription_start_site +1000
end <- tss_info.Mm.Nrl$transcription_start_site - 1000

#Flipping start and end...
tss_range.Mm.Nrl <- GRanges(seqnames = paste0("chr", tss_info.Mm.Nrl$chromosome_name),
                           ranges = IRanges(start = end, end = start),
                           strand = ifelse(tss_info.Mm.Nrl$strand == 1, "+", "-"))

remove(start)
remove(end)

DNA.Mm.Nrl <- getSeq(BSgenome.Mmusculus.UCSC.mm39, tss_range.Mm.Nrl)
head(DNA.Mm.Nrl)
```

```
## DNASTringSet object of length 1:
##      width seq
## [1] 2001 CATAACACTTGACTCACTACAGGCAATACATGAT...CTTTTTTTTTTTTTTTTTTCGAGACAGGGTTTCT
```

Step 2: plotting using ggBio

Need to convert the sequences into characters to parse for any occurrence of CG.

```
#Convert sequence to a character vector
dna_str <- as.character(DNA.Hs.NRL)
bases <- strsplit(dna_str, "")[[1]]

#Create a status vector: default to "Other" to separate out the "CG"s
```

```

color_status <- rep("Other", length(bases))

#Identify CpGs
for (i in 1:(length(bases) - 1)) {
  if (bases[i] == "C" && bases[i+1] == "G") {
    color_status[i] <- "CG"
    color_status[i+1] <- "CG"
  }
}

#Expand your TSS range into 1-bp intervals
gr_expanded.HN <- unlist(tile(tss_range.Hs.NRL, width = 1))

#Add the new metadata column
mcols(gr_expanded.HN)$is_cg <- color_status

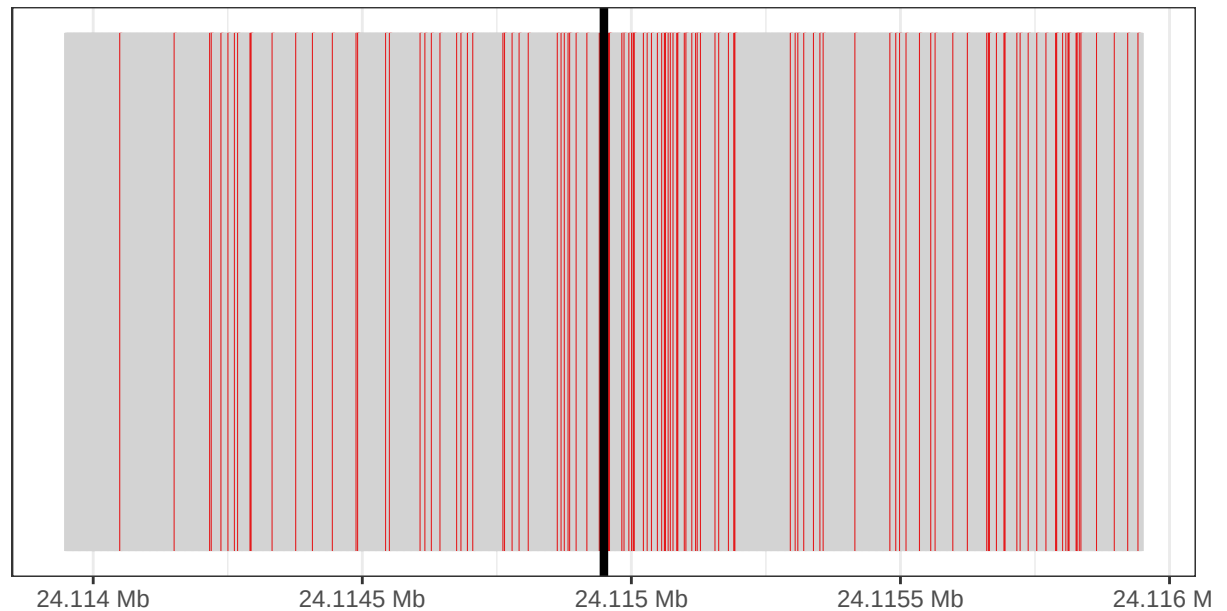
Hs.NRL.plot <- autoplot(gr_expanded.HN, geom = "rect", aes(fill = is_cg, color = is_cg)) +
  geom_vline(aes(xintercept = tss_info.Hs.NRL$transcription_start_site), color = "black", linewidth = 1)
  scale_fill_manual(values = c("CG" = "#E41A1C",
                                "Other" = "#D3D3D3")) +
  scale_color_manual(values = c("CG" = "#E41A1C",
                                "Other" = "#D3D3D3")) +
  labs(color = "CpG",
       fill = "CpG",
       title = "Human NRL") +
  theme_bw() +
  theme(
    plot.title = element_text(face = "bold",
                              size = 20,
                              hjust = 0.5,
                              margin = margin(b = 15, unit = "pt")),
    legend.text = element_text(size = 15),
    legend.title = element_text(size = 17,
                                face = "bold"),
    legend.position = "top"
  )

Hs.NRL.plot

```

Human NRL

CpG ■ CG ■ Other



Hs NRL

```
#Convert sequence to a character vector
dna_str <- as.character(DNA.Hs.THRB)
bases <- strsplit(dna_str, "")[[1]]

#Create a status vector
color_status <- rep("Other", length(bases))

# Identify CpGs
for (i in 1:(length(bases) - 1)) {
  if (bases[i] == "C" && bases[i+1] == "G") {
    color_status[i] <- "CG"
    color_status[i+1] <- "CG"
  }
}

#Expand your TSS range into 1-bp intervals
gr_expanded.HT <- unlist(tile(tss_range.Hs.THRB, width = 1))

#Add the new metadata column
mcols(gr_expanded.HT)$is_cg <- color_status

#Autoplot using ggbio
Hs.THRB.plot <- autoplot(gr_expanded.HT, geom = "rect", aes(fill = is_cg, color = is_cg)) +
```

```

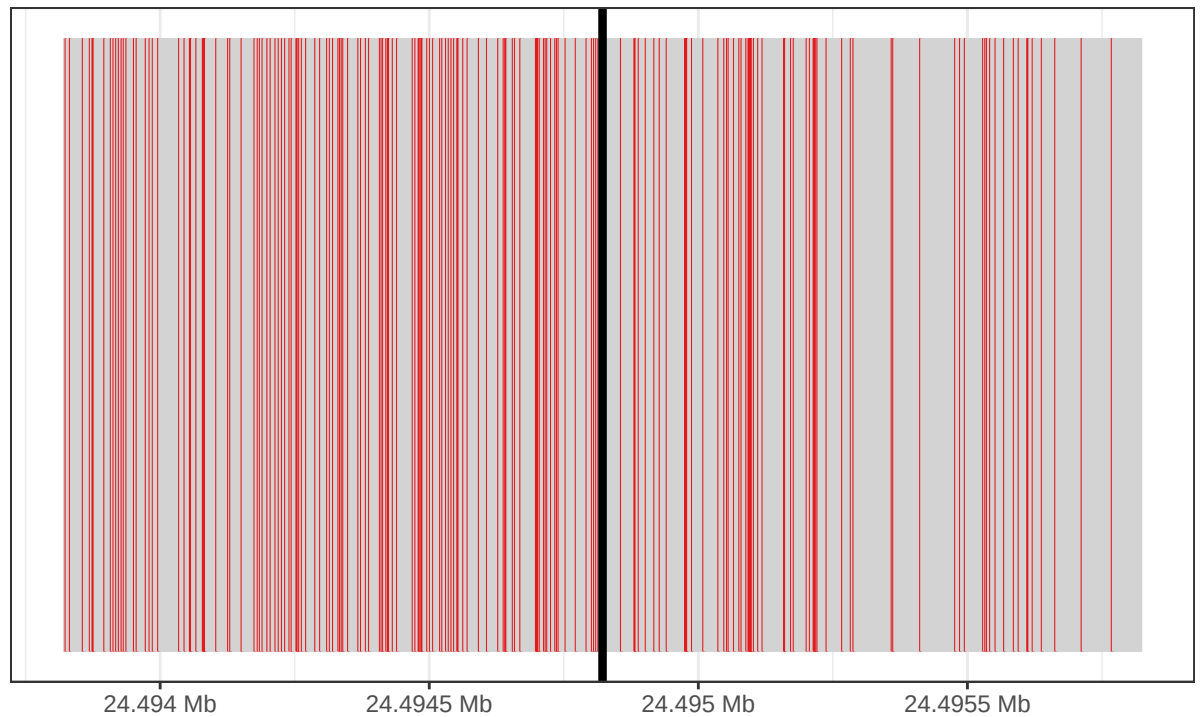
geom_vline(aes(xintercept = tss_info.Hs.THRB$transcription_start_site), color = "black", linewidth = 1) +
scale_fill_manual(values = c("CG" = "#E41A1C",
                             "Other" = "#D3D3D3")) +
scale_color_manual(values = c("CG" = "#E41A1C",
                              "Other" = "#D3D3D3")) +
labs(color = "CpG",
     fill = "CpG",
     title = "Human THRB") +

theme_bw() +
theme(
  plot.title = element_text(face = "bold",
                             size = 20,
                             hjust = 0.5,
                             margin = margin(b = 15, unit = "pt")),
  legend.text = element_text(size = 15),
  legend.title = element_text(size = 17,
                               face = "bold"),
  legend.position = "none"
)

```

Hs.THRB.plot

Human THRB



Hs THRB

```

#Convert sequence to a character vector
dna_str <- as.character(DNA.Mm.Nrl)
bases <- strsplit(dna_str, "")[[1]]

#Create a status vector
color_status <- rep("Other", length(bases))

#Identify CpGs
for (i in 1:(length(bases) - 1)) {
  if (bases[i] == "C" && bases[i+1] == "G") {
    color_status[i] <- "CG"
    color_status[i+1] <- "CG"
  }
}

#Expand your TSS range into 1-bp intervals
gr_expanded.MN <- unlist(tile(tss_range.Mm.Nrl, width = 1))

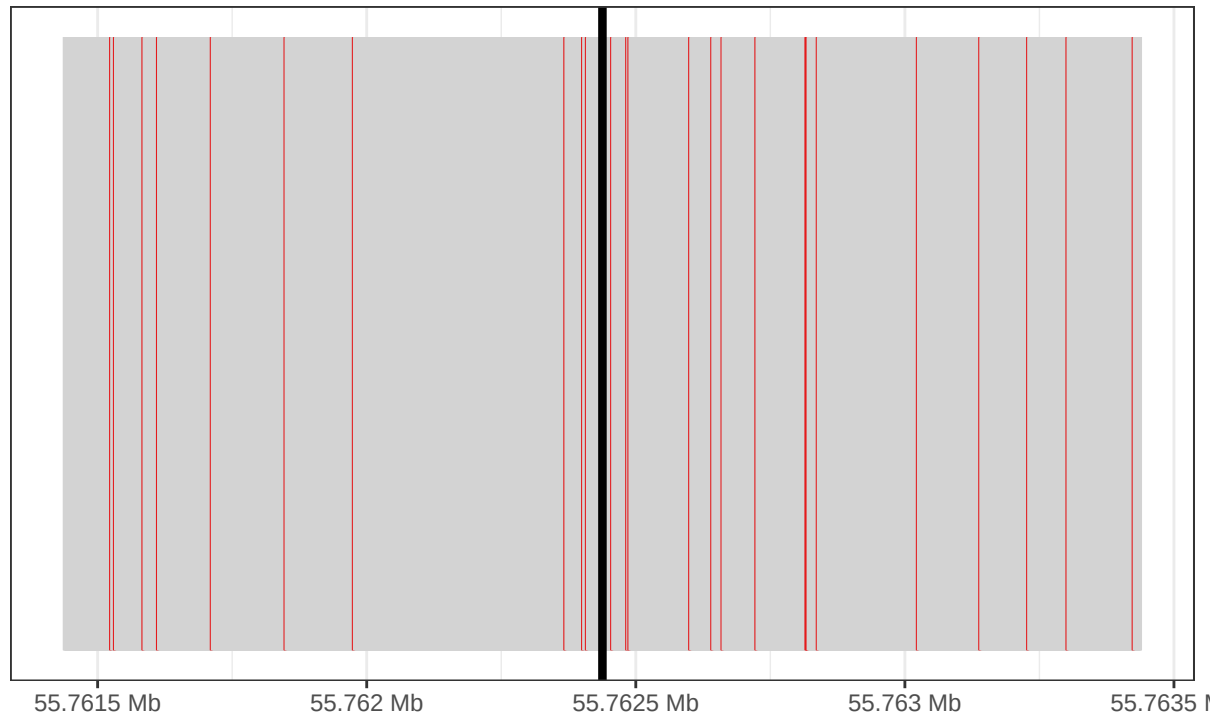
#Add the new metadata column
mcols(gr_expanded.MN)$is_cg <- color_status

Mm.Nrl.plot <- autoplot(gr_expanded.MN, geom = "rect", aes(fill = is_cg, color = is_cg)) +
  geom_vline(aes(xintercept = tss_info.Mm.Nrl$transcription_start_site), color = "black", linewidth = 1)
  scale_fill_manual(values = c("CG" = "#E41A1C",
                                "Other" = "#D3D3D3")) +
  scale_color_manual(values = c("CG" = "#E41A1C",
                                "Other" = "#D3D3D3")) +
  labs(color = "CpG",
       fill = "CpG",
       title = "Mouse NRL") +
  theme_bw() +
  theme(
    plot.title = element_text(face = "bold",
                              size = 20,
                              hjust = 0.5,
                              margin = margin(b = 15, unit = "pt")),
    legend.text = element_text(size = 15),
    legend.title = element_text(size = 17,
                                face = "bold"),
    legend.position = "none"
  )

Mm.Nrl.plot

```


Mouse NRL



Mm NRL

```
#Convert sequence to a character vector
dna_str <- as.character(DNA.Mm.Thrb)
bases <- strsplit(dna_str, "")[[1]]

#Create a status vector
color_status <- rep("Other", length(bases))

#Identify CpGs
for (i in 1:(length(bases) - 1)) {
  if (bases[i] == "C" && bases[i+1] == "G") {
    color_status[i] <- "CG"
    color_status[i+1] <- "CG"
  }
}

#Expand your TSS range into 1-bp intervals
gr_expanded.MT <- unlist(tile(tss_range.Mm.Thrb, width = 1))

#Add the new metadata column
mcols(gr_expanded.MT)$is_cg <- color_status

Mm.Thrb.plot <- autoplot(gr_expanded.MT, geom = "rect", aes(fill = is_cg, color = is_cg)) +
  geom_vline(aes(xintercept = tss_info.Mm.Thrb$transcription_start_site), color = "black", linewidth = 1)
```

```

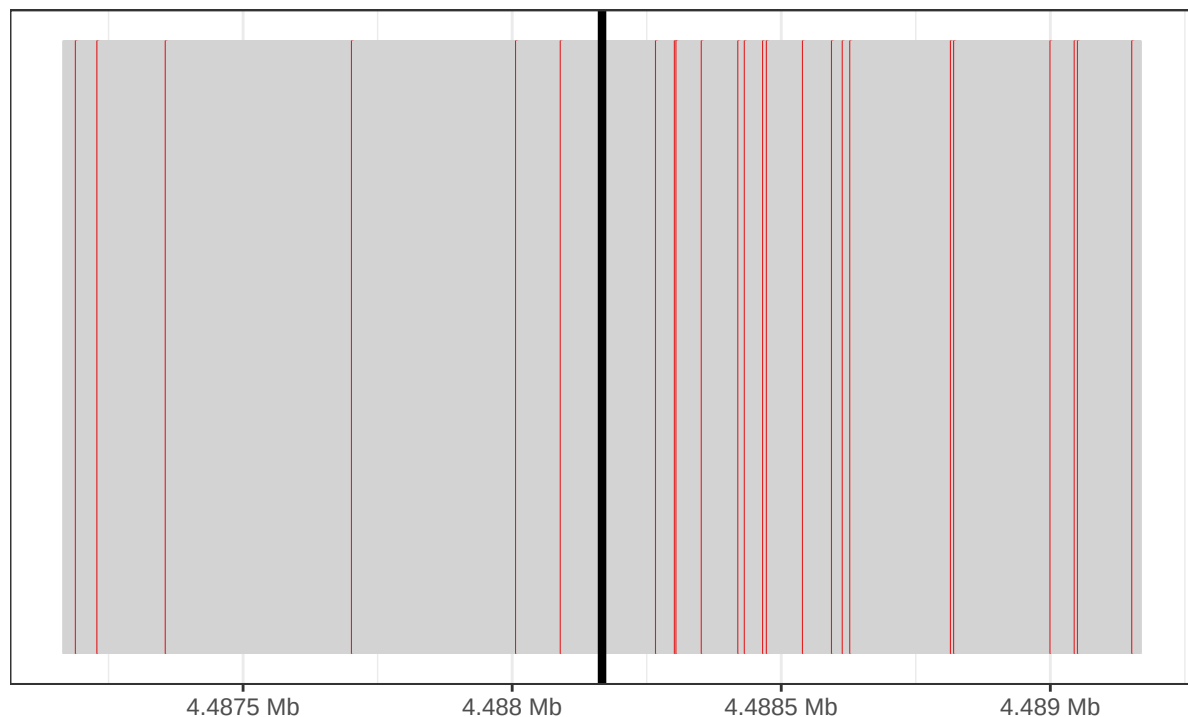
scale_fill_manual(values = c("CG" = "#E41A1C",
                             "Other" = "#D3D3D3")) +
scale_color_manual(values = c("CG" = "#E41A1C",
                              "Other" = "#D3D3D3")) +

labs(color = "CpG",
     fill = "CpG",
     title = "Mouse THRB") +
theme_bw() +
theme(
  plot.title = element_text(face = "bold",
                             size = 20,
                             hjust = 0.5,
                             margin = margin(b = 15, unit = "pt")),
  legend.text = element_text(size = 15),
  legend.title = element_text(size = 17,
                               face = "bold"),
  legend.position = "none"
)

```

Mm.Thrb.plot

Mouse THRB



Mm THRB

Step 3: Combining all

Combining all the plots using `gridExtra` to account for the `ggbio` objects.

```

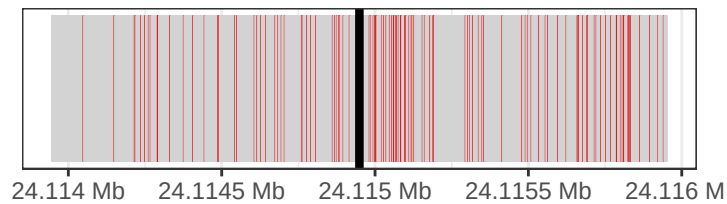
#Converting ggbio objects to ggplot to use with gridExtra
p1 <- Hs.NRL.plot@ggplot
p2 <- Mm.Nrl.plot@ggplot
p3 <- Hs.THRB.plot@ggplot
p4 <- Mm.Thrb.plot@ggplot

#Combine with collected legends so the sizes are all the same
full.cpg<- wrap_plots(p1, p2, p3, p4, ncol = 1, heights = c(2, 2, 2, 2)) +
  plot_layout(guides = "collect") &
  theme(legend.position = "bottom")

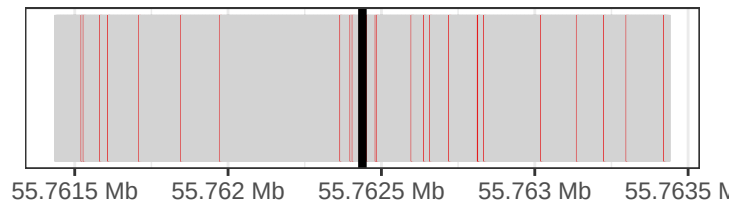
full.cpg

```

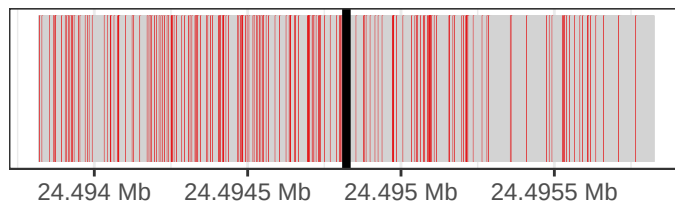
Human NRL



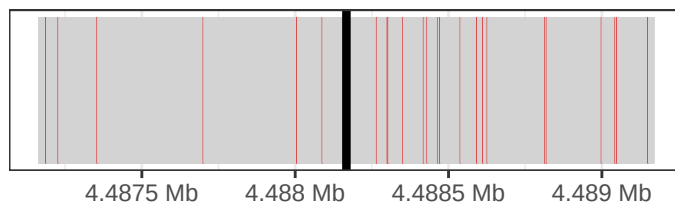
Mouse NRL



Human THRB



Mouse THRB



CpG ■ **CG** ■ **Other**

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)  
## Platform: x86_64-w64-mingw32/x64
```

```

## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] ggbio_1.58.0           BSgenome.Mmusculus.UCSC.mm39_1.4.3
## [3] BSgenome.Hsapiens.UCSC.hg38_1.4.5 BSgenome_1.78.0
## [5] rtracklayer_1.70.1     BiocIO_1.20.0
## [7] Biostrings_2.78.0      XVector_0.50.0
## [9] GenomeInfoDb_1.46.2    org.Gg.eg.db_3.22.0
## [11] org.Mm.eg.db_3.22.0    org.Hs.eg.db_3.22.0
## [13] biomaRt_2.66.1         GenomicFeatures_1.62.0
## [15] AnnotationDbi_1.72.0   Biobase_2.70.0
## [17] GenomicRanges_1.62.1  Seqinfo_1.0.0
## [19] IRanges_2.44.0         S4Vectors_0.48.0
## [21] BiocGenerics_0.56.0    generics_0.1.4
## [23] BiocManager_1.30.27    gridExtra_2.3
## [25] patchwork_1.3.2        lubridate_1.9.5
## [27] forcats_1.0.1          stringr_1.6.0
## [29] dplyr_1.1.4            purrr_1.2.1
## [31] readr_2.1.6            tidyr_1.3.2
## [33] tibble_3.3.1           ggplot2_4.0.3
## [35] tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] RColorBrewer_1.1-3      rstudioapi_0.18.0
## [3] jsonlite_2.0.0          magrittr_2.0.4
## [5] farver_2.1.2            rmarkdown_2.31
## [7] vctrs_0.7.1             memoise_2.0.1
## [9] Rsamtools_2.26.0        RCurl_1.98-1.17
## [11] base64enc_0.1-6         tinytex_0.59
## [13] htmltools_0.5.9         S4Arrays_1.10.1
## [15] progress_1.2.3          curl_7.0.0
## [17] SparseArray_1.10.8      Formula_1.2-5
## [19] htmlwidgets_1.6.4       plyr_1.8.9
## [21] httr2_1.2.2             cachem_1.1.0
## [23] GenomicAlignments_1.46.0 lifecycle_1.0.5
## [25] pkgconfig_2.0.3         Matrix_1.7-3
## [27] R6_2.6.1                fastmap_1.2.0

```

## [29]	MatrixGenerics_1.22.0	digest_0.6.39
## [31]	colorspace_2.1-2	OrganismDbi_1.52.0
## [33]	Hmisc_5.2-5	RSQLite_2.4.6
## [35]	filelock_1.0.3	timechange_0.4.0
## [37]	httr_1.4.8	abind_1.4-8
## [39]	compiler_4.5.0	bit64_4.6.0-1
## [41]	withr_3.0.2	htmlTable_2.5.0
## [43]	S7_0.2.1	backports_1.5.1
## [45]	BiocParallel_1.44.0	DBI_1.3.0
## [47]	rappdirs_0.3.4	DelayedArray_0.36.0
## [49]	rjson_0.2.23	tools_4.5.0
## [51]	foreign_0.8-90	otel_0.2.0
## [53]	nnet_7.3-20	glue_1.8.0
## [55]	restfulr_0.0.16	grid_4.5.0
## [57]	checkmate_2.3.4	cluster_2.1.8.1
## [59]	reshape2_1.4.5	gtable_0.3.6
## [61]	tzdb_0.5.0	ensembldb_2.34.0
## [63]	data.table_1.18.2.1	hms_1.1.4
## [65]	xml2_1.5.2	pillar_1.11.1
## [67]	BiocFileCache_3.0.0	lattice_0.22-7
## [69]	bit_4.6.0	biovizBase_1.58.0
## [71]	RBGL_1.86.0	tidyselect_1.2.1
## [73]	knitr_1.51	ProtGenerics_1.42.0
## [75]	SummarizedExperiment_1.40.0	xfun_0.56
## [77]	matrixStats_1.5.0	stringi_1.8.7
## [79]	UCSC.utils_1.6.1	lazyeval_0.2.3
## [81]	yaml_2.3.12	evaluate_1.0.5
## [83]	codetools_0.2-20	cigarillo_1.0.0
## [85]	graph_1.88.1	cli_3.6.5
## [87]	rpart_4.1.24	dichromat_2.0-0.1
## [89]	Rcpp_1.1.1-1.1	dbplyr_2.5.2
## [91]	png_0.1-8	XML_3.99-0.22
## [93]	parallel_4.5.0	blob_1.3.0
## [95]	prettyunits_1.2.0	AnnotationFilter_1.34.0
## [97]	bitops_1.0-9	VariantAnnotation_1.56.0
## [99]	scales_1.4.0	crayon_1.5.3
## [101]	rlang_1.1.7	KEGGREST_1.50.0